

EE 475 Introduction to Image Processing

Homework 03

Boran Aybak Kilic
Student ID: 2022401207

1 a) Creating a Binary Letter 'T' Image

To construct the binary image, we first define the dimensions of the canvas. The image height H is set to 196 pixels and the width W is set to 128 pixels. A logical array (which holds only 1s and 0s) is initialized with all zeros.

```
1 H = 196;  
2 W = 128;  
3 T_height = 40;  
4 T_top_width = 25;  
5 T_stroke = 5;
```

Listing 1: Defining image dimensions.

To center the 'T' perfectly, we calculate a **geometric center** based on continuous coordinates, rather than discrete pixel indices. This is a more general approach that correctly finds the middle of the canvas, regardless of whether the dimensions H and W are even or odd. This method yields $c_y = 98.5$ and $c_x = 64.5$.

```
1 c_y = (H + 1) / 2; % 98.5  
2 c_x = (W + 1) / 2; % 64.5
```

Listing 2: Calculating the continuous geometric center.

The 'T' shape is then constructed in two parts: a horizontal top bar [3](#) and a vertical stem [4](#). The coordinates for these parts are calculated relative to the continuous center (c_x, c_y) . When these continuous coordinates are used to define a pixel's start or end, they must be converted back to an integer index. The `round()` function is used for this purpose, finding the nearest pixel index to the "ideal" geometric boundary.

```
1 % 1. Top Bar (25 pixels wide, 5 pixels high)  
2 y_start_top = round(c_y - T_height / 2); % 79  
3 y_end_top = y_start_top + T_stroke - 1; % 83  
4 x_start_top = round(c_x - T_top_width / 2); % 52  
5 x_end_top = x_start_top + T_top_width - 1; % 76  
6  
7 binary_T(y_start_top:y_end_top, x_start_top:x_end_top) = 1;
```

Listing 3: Code for determining the coordinates of 4 corners of the rectangular top bar of letter T

When defining the end coordinates, we add the total desired width (e.g., `T_stroke` or `T_top_width`) and then **subtract 1**. This adjustment is essential for achieving the correct pixel dimensions.

In MATLAB, an array index `start:end` includes both `start` and `end`, so the total number of elements is `end - start + 1`. To get a specific width, we must set `end = start + width - 1`. Forgetting the `-1` would result in a shape that is one pixel wider or taller than intended.

```

1 % 2. Stem (5 pixels wide, remaining height)
2 % Stem height is 40 (total) - 5 (top bar) = 35
3 y_start_stem = y_end_top + 1;           % 84
4 y_end_stem = y_start_top + T_height - 1 ; % 118
5 x_start_stem = round(c_x - T_stroke / 2); % 62
6 x_end_stem = x_start_stem + T_stroke - 1; % 66
7
8 binary_T(y_start_stem:y_end_stem, x_start_stem:x_end_stem) = 1;

```

Listing 4: Code for determining the coordinates of 4 corners of the rectangular vertical stem of letter T

Finally, these calculated start and end indices are used to select regions of the `binary_T` matrix and set their values to 1 (true), creating the white 'T' shape on the black background.

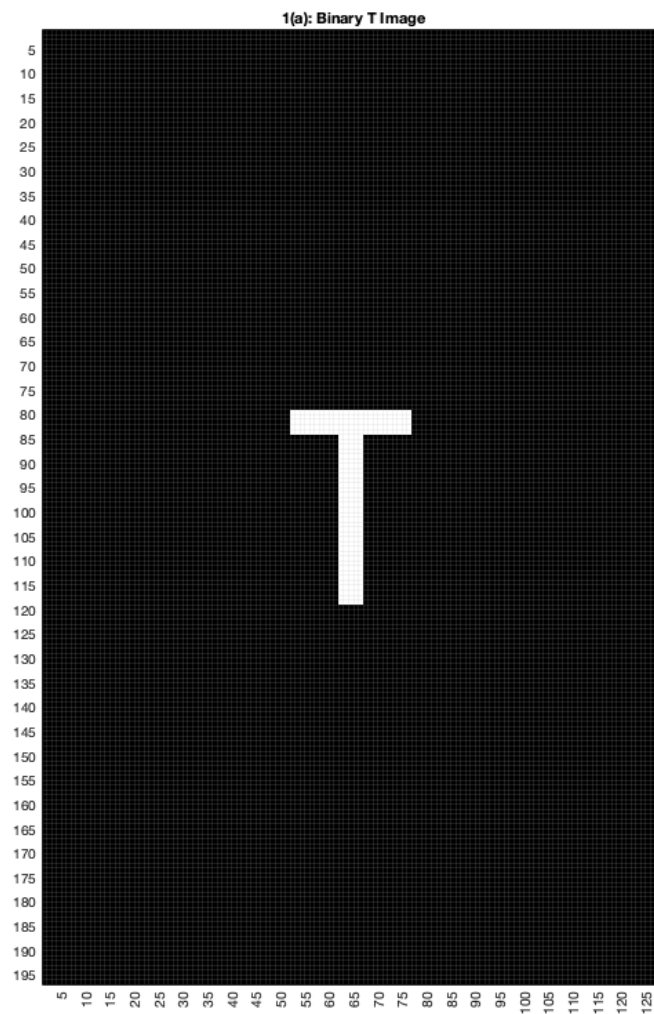


Figure 1: The generated binary 'T' image, plotted with a visible pixel grid and 5-pixel enumeration.

1 b) Creating a Distance-Based 'T' Image

This part requires the pixels of the 'T' to have a `uint8` value equal to their Euclidean distance from the image center. The core of this task involves creating a full-image distance matrix and then using the `binary_T` image from part (a) as a mask.

First, we use `meshgrid` to create matrices `X` and `Y` containing the column and row coordinates of every pixel before creating `DistMatrix` which holds the Euclidean distance of every pixel to the center of the image. We then initialize a `uint8` output image with all zeros. The key step is assigning the distance values only at the locations included in the letter 'T'. To achieve that we use the `binary_T` from the previous option as a logical mask.

```
1 % 1. Create coordinate grids (X = columns, Y = rows)
2 [X, Y] = meshgrid(1:W, 1:H);
3
4 % 2. Calculate Euclidean distance (DistMatrix definition discussed below)
5 DistMatrix = sqrt((X - c_x).^2 + (Y - c_y).^2) % See discussion
6
7 % 3. Initialize the output image as uint8
8 distance_T = zeros(H, W, 'uint8');
9
10 % 4. Assign rounded distance values only to the 'T' pixels
11 % We use the logical mask 'binary_T' from part 1(a)
12 % We round the distance to get an integer value suitable for uint8
13 distance_T(binary_T) = uint8(round(DistMatrix(binary_T)));
```

Listing 5: General code for distance-based 'T' creation.

A critical consideration is the definition of the "center of the image," as the 196×128 dimensions are both even. This leads to two primary approaches.

Approach 1: True Geometric Center The true geometric center is at the continuous coordinates $(c_x, c_y) = ((W + 1)/2, (H + 1)/2) = (64.5, 98.5)$. In the pixel index system (where integers are pixel centers), these half-integer values represent the **border** between pixels. We can calculate the distance of each pixel to this precise point.

```
1 % Approach 1: True geometric center (64.5, 98.5)
2 DistMatrix_geo = sqrt((X - c_x).^2 + (Y - c_y).^2);
```

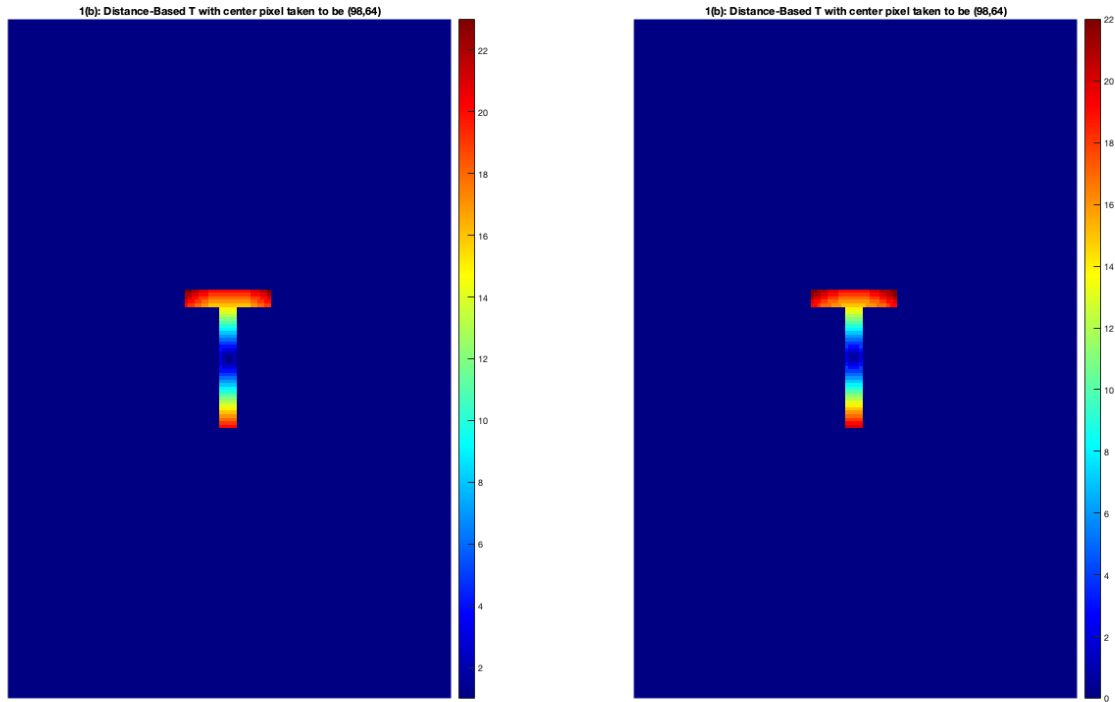
Listing 6: Distance calculation using the true geometric center.

Approach 2: Floored (Pixel-Aligned) Center Alternatively, we can select a specific pixel index to serve as the center by flooring the geometric coordinates: $(c'_x, c'_y) = (\lfloor c_x \rfloor, \lfloor c_y \rfloor) = (64, 98)$. This choice has a significant advantage: it aligns with the construction of our 'T'. The 5-pixel-wide stem was built around the 64th column (indices 62-66), making column 64 its center. Choosing the pixel (98, 64) as the origin for the distance calculation results in a perfectly **symmetric distribution of distances** across the 'T' shape, as visualized in Figure 2.

```
1 % Approach 2: Floored center (64, 98)
2 DistMatrix_floor = sqrt((X - floor(c_x)).^2 + (Y - floor(c_y)).^2);
```

Listing 7: Distance calculation using the floored center.

Due to the desirable symmetric properties, we will adopt **Approach 2** for this report. The maximum possible distance (from (64, 98) to (128, 196)) is ≈ 117.1 , which fits perfectly within the 0-255 range of the `uint8` data type after rounding.



(a) Distance distribution using Approach 1 (Geometric Center).

(b) Distance distribution using Approach 2 (Floored Center).

Figure 2: A side-by-side comparison of the distance values on the 'T' for both center definitions. Approach 2 (right) shows clear symmetry.

1 c) Image Rotation with Interpolation

For this task, the `imrotate()` function from the Signal Processing Toolbox in MATLAB could be used and would provide a valid answer. However, to acquire the necessary skills targeted in this homework, we decided to implement the rotation manually. This approach allows for a deeper understanding of the underlying geometry and a clear, explicit choice of interpolation method.

Forward vs. Inverse Mapping In the context of affine image transformations like rotation, a "forward mapping" approach would iterate through each pixel of the *input* image, calculate its new rotated coordinate, and "drop" its value into the *output* image. This method has significant problems. Multiple input pixels may map to the same output coordinate, creating ambiguity about which value to use. More importantly, it often leaves "holes" or gaps in the output image, where no input pixel has landed.

Hence, we opted for **inverse mapping**. This approach resolves all ambiguities. We iterate through every pixel (x_{out}, y_{out}) of the *output* grid and calculate its corresponding source coordinate (x_{in}, y_{in}) in the *input* image.

Our workflow for **Inverse Mapping** is as follows:

1. **Start with an empty output image:** We create a new 196×128 matrix, `rotated.T`, filled with zeros.

2. **Define Rotation Center:** We use the geometric center coordinates defined in part (a): $c_x = 64.5$ and $c_y = 98.5$.

3. **Define Inverse Transformation:** For a rotation by angle θ , we map each output coordinate (x_{out}, y_{out}) back to a source coordinate (x_{in}, y_{in}) . This requires an inverse transformation. We first translate the output coordinate to the origin, apply the rotation, and then translate back.

A key point is the form of the rotation matrix. In standard Cartesian coordinates, an inverse mapping would use the inverse (clockwise) rotation matrix. However, in MATLAB's image coordinate system, the y-axis is inverted (increasing y goes down). To account for this, the standard counter-clockwise (CCW) rotation equations are used, as this correctly performs the inverse mapping from the output grid back to the source grid.

The inverse mapping equations for a source coordinate (x_{in}, y_{in}) given an output coordinate (x_{out}, y_{out}) are:

$$x_{in} = (x_{out} - c_x) \cos(\theta) - (y_{out} - c_y) \sin(\theta) + c_x \quad (1)$$

$$y_{in} = (x_{out} - c_x) \sin(\theta) + (y_{out} - c_y) \cos(\theta) + c_y \quad (2)$$

Note that since the rotation is performed around the origin, we need to adjust pixel coordinates so that center pixel maps to origin. We add this shift back after the rotation, to go back from Cartesian domain into image domain.

4. **Interpolate:** The calculated source coordinate (x_{in}, y_{in}) will almost always be a non-integer (e.g., (55.3, 81.7)). We cannot simply read the value at this location. This is where interpolation is required. We use MATLAB's `interp2` function to sample the value from the original `binary_T` image at this fractional coordinate.

5. **Assign Value:** The interpolated value is then assigned to the output pixel:
`rotated_T(y_out, x_out) = interpolated_value.`

Implementation This entire process is implemented efficiently without loops by using `meshgrid` to perform the transformation on all pixels at once.

```

1 test_angle_deg = 35;
2 interpolation_method = 'linear'; % 'linear' = bilinear, 'cubic' = bicubic, '
   nearest'
3 theta = deg2rad(test_angle_deg);
4
5 % --- 2. Create coordinate grids ---
6 [X_in, Y_in] = meshgrid(1:W, 1:H);
7 [X_out, Y_out] = meshgrid(1:W, 1:H);
8
9 % --- 3. Define the CCW Inverse Rotation Transformation ---
10 % This maps the output grid (X_out, Y_out) back to the
11 % corresponding coordinates in the input image (X_in_mapped, Y_in_mapped)
12 X_centered = X_out - c_x;
13 Y_centered = Y_out - c_y;
14
15 X_in_mapped = X_centered * cos(theta) - Y_centered * sin(theta) + c_x;
16 Y_in_mapped = X_centered * sin(theta) + Y_centered * cos(theta) + c_y;
17
18 % --- 4. Interpolate the original image ---
19 % We use double for interpolation to handle fractional values
20 original_image_data = double(binary_T);
21
22 rotated_image = interp2(X_in, Y_in, original_image_data, ...
23   X_in_mapped, Y_in_mapped, ...

```

```

24         interpolation_method);
25
26 % --- 5. Clean up the result ---
27 % Set pixels that mapped to "outside" the original image to 0
28 rotated_image(isnan(rotated_image)) = 0;
29 % Set any negative-valued interpolation artifacts to 0
30 rotated_image(rotated_image < 0) = 0;

```

Listing 8: Code for manual image rotation using inverse mapping and interpolation.

In the code, we first convert the angle θ from degrees to radians. We then create coordinate grids for the original image (X_{in} , Y_{in}) and the output image (X_{out} , Y_{out}).

The lines `X_in_mapped` and `Y_in_mapped` apply the inverse transformation [1](#) to the entire output grid in a single vectorized operation.

The core of the operation is the `interp2` function. It is given the original grid (X_{in} , Y_{in}) and the original data (`original_image_data`), and it is asked to find the values at the newly calculated non-integer coordinates (`X_in_mapped`, `Y_in_mapped`) using the specified `interpolation_method`.

Finally, two crucial cleanup steps are performed. The line `rotated_image(isnan(rotated_image)) = 0;` handles pixels in the output image (e.g., at the corners) whose source coordinates mapped to *outside* the bounds of the original image. `interp2` returns NaN for these locations. We set these NaN values to 0, ensuring the rotated 'T' is placed on a black background and the image size is maintained.

Additionally, `rotated_image(rotated_image < 0) = 0;` is used since high-order interpolation methods like 'cubic' can "overshoot" at sharp edges (the 0-to-1 transition), resulting in small negative values. This line clips any such non-feasible values back to 0.

1 d) Image Rotation Results and Analysis

In this section, we present the results of rotating both the binary 'T' (from part 1a) and the distance-based 'T' (from part 1b) by various angles: 35° , 60° , 90° , 150° , and -40° . For each rotation, we compare the output generated using nearest neighbor, bilinear, and bicubic interpolation methods. The results for the binary 'T' are shown in Figures [3](#) to [7](#), and for the distance-based 'T' in Figures [8](#) to [12](#).

We can make some observations looking at the the resulting figures:

1. **Nearest Neighbor Interpolation:** As seen in the leftmost subplot of each figure (e.g., Figures [3a](#) and [8a](#)), nearest neighbor interpolation produces the most aliased or "grainy" results. Edges, especially diagonal lines, appear visibly pixelated and staircase-like. This is because each output pixel is simply assigned the value of the single closest input pixel, resulting in hard, abrupt transitions.
2. **Bilinear and Bicubic Interpolation:** These methods produce significantly smoother rotations compared to nearest neighbor. Bilinear interpolation (center subplot) calculates the output pixel value as a weighted average of the four nearest input pixels, leading to smoother transitions. Bicubic interpolation (rightmost subplot), which uses a weighted average of sixteen nearest input pixels, offers a slightly more sophisticated and often visually smoother result, especially evident in the subtle gradations on edges (e.g., compare Figures [3b](#) and [3c](#)). However, the visual difference between bilinear and bicubic can be subtle for simple binary images.
3. **Overshoot in Distance-Based 'T' Images:** When rotating the `distance_T` images, which have varying pixel values not just at the edges but across the entire 'T' (as shown in Figure [8](#)), the phenomenon of "overshoot" or "undershoot" is more apparent with higher-order interpolation methods (bilinear and bicubic). This can lead to interpolated pixel

values slightly exceeding the maximum or falling below the minimum valid range if not explicitly clipped (as handled by `rotated_image(rotated_image < 0) = 0;`).

4. **Finite Size Effects:** Despite the effectiveness of bilinear and bicubic interpolation in smoothing transitions, diagonal edges still exhibit some degree of pixelation across all methods. This is largely due to the small finite dimensions of our image (196×128) and, more importantly, the small size of the letter 'T' itself. At this resolution, individual pixels remain distinguishable by the human eye. These finite size effects would be significantly less apparent as the image dimensions and the size of the object within the image increase, allowing for finer interpolation and smoother visual results.

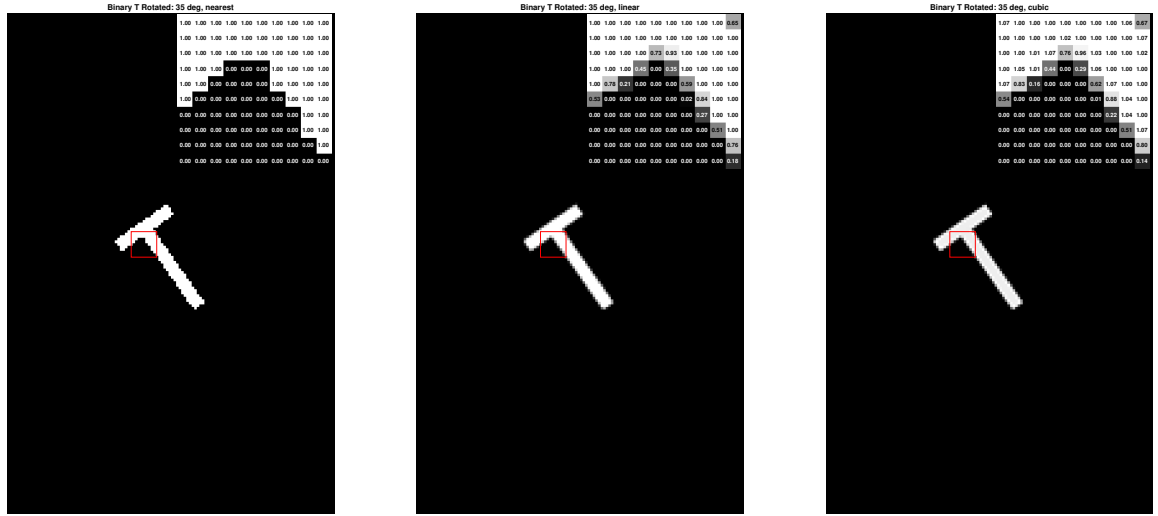


Figure 3: Binary 'T' rotated by 35°. Left: Nearest Neighbor, Middle: Bilinear, Right: Bicubic interpolation.

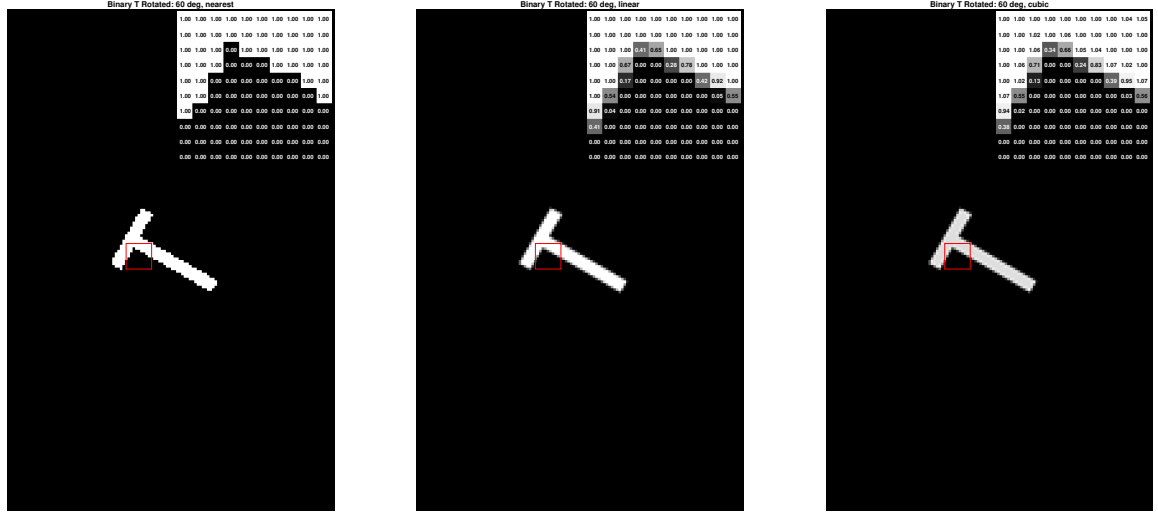


Figure 4: Binary 'T' rotated by 60°. Left: Nearest Neighbor, Middle: Bilinear, Right: Bicubic interpolation.

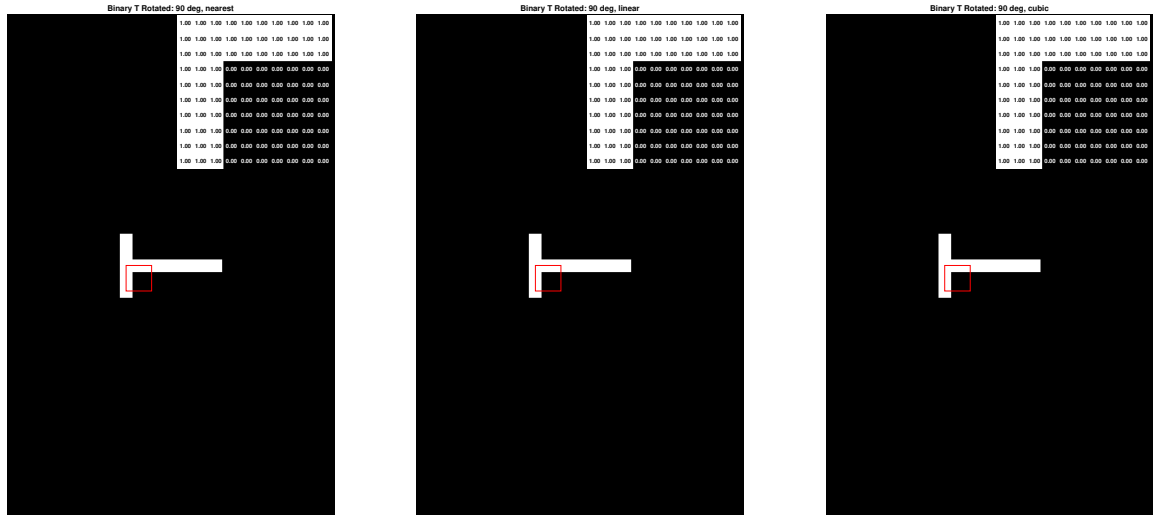


Figure 5: Binary 'T' rotated by 90°. Left: Nearest Neighbor, Middle: Bilinear, Right: Bicubic interpolation.

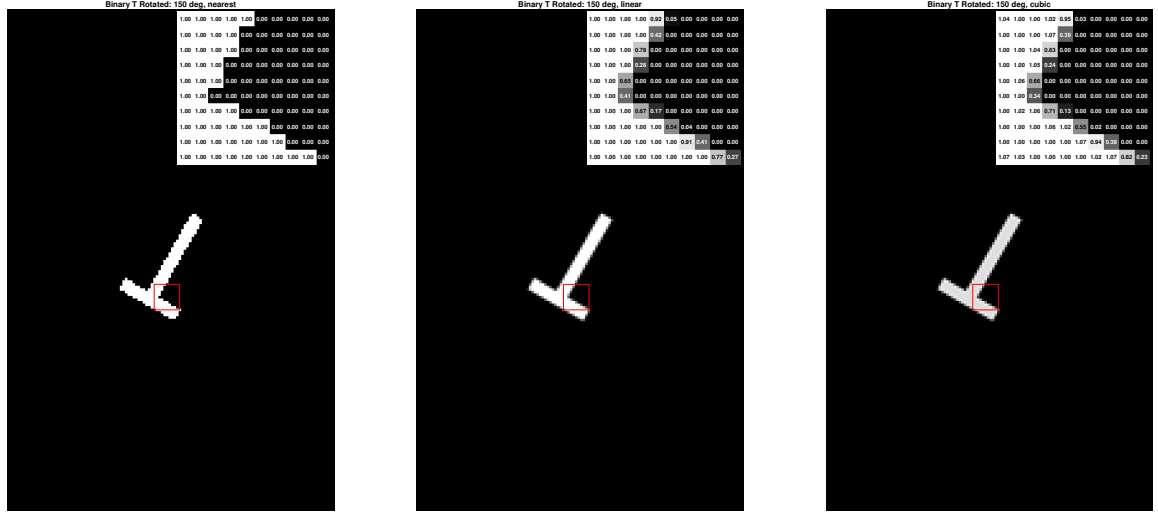


Figure 6: Binary 'T' rotated by 150° . Left: Nearest Neighbor, Middle: Bilinear, Right: Bicubic interpolation.

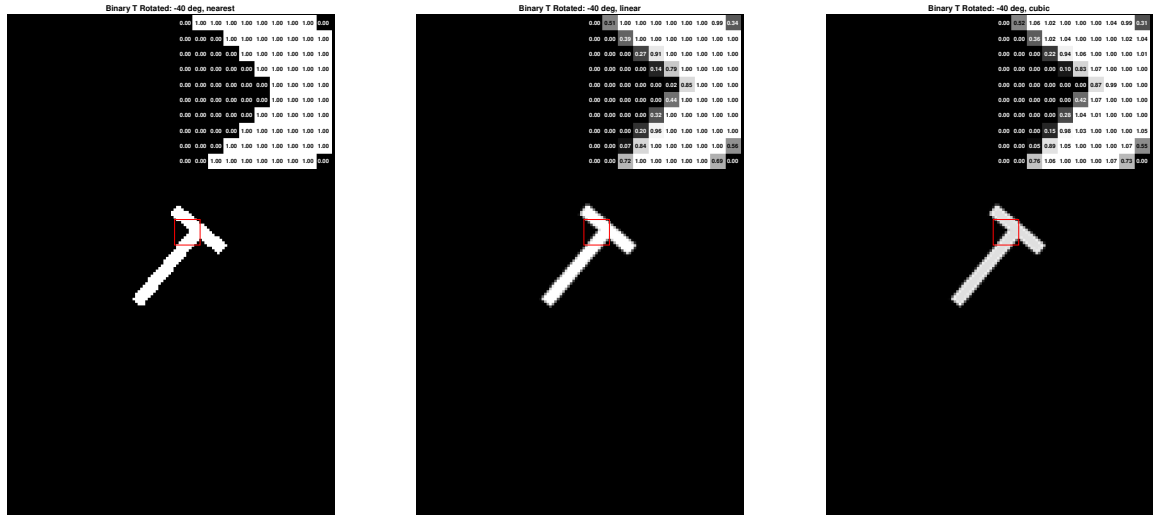


Figure 7: Binary 'T' rotated by -40° . Left: Nearest Neighbor, Middle: Bilinear, Right: Bicubic interpolation.

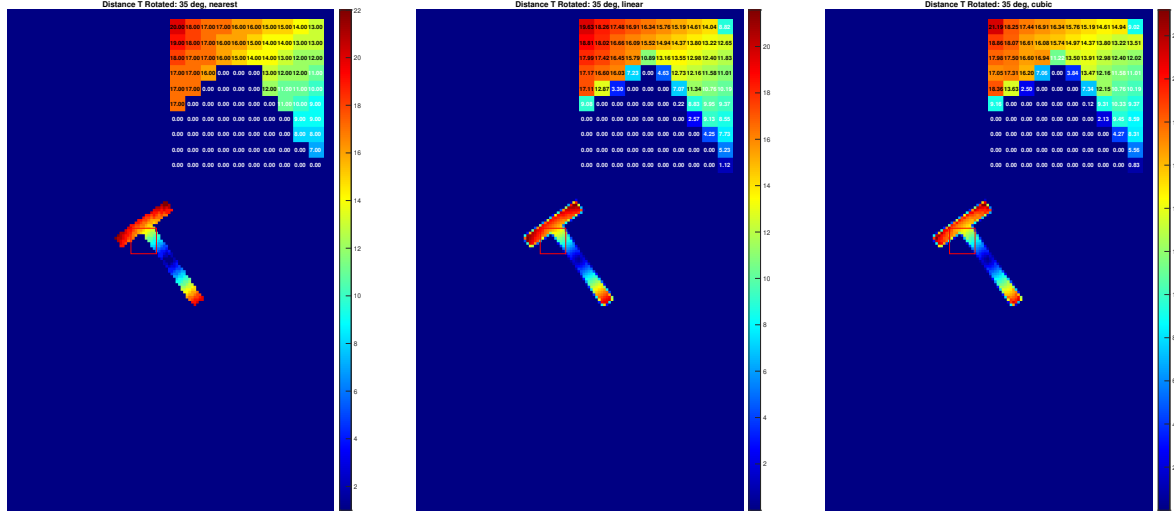


Figure 8: Distance-based 'T' rotated by 35°. Left: Nearest Neighbor, Middle: Bilinear, Right: Bicubic interpolation.

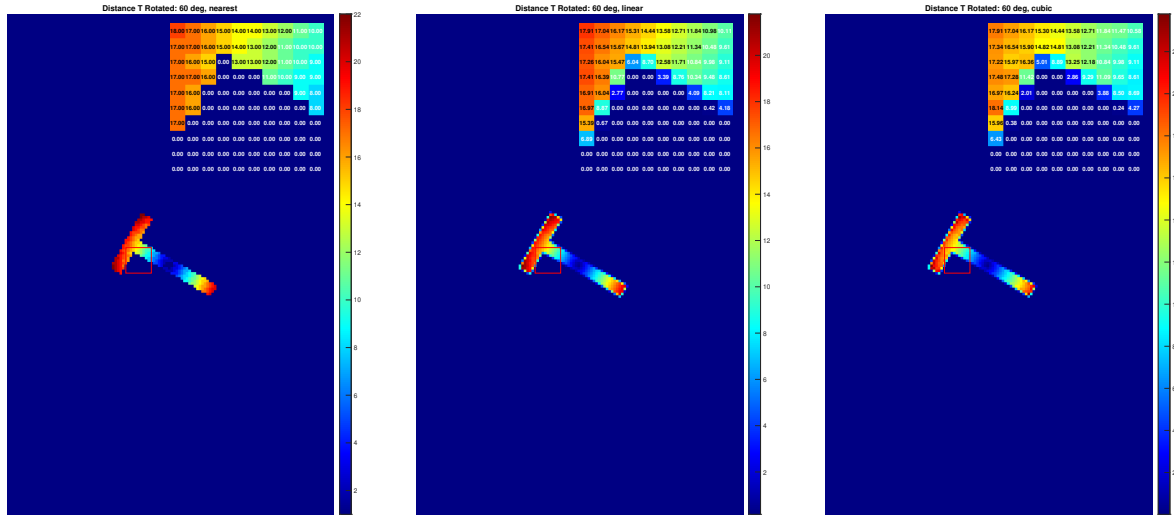


Figure 9: Distance-based 'T' rotated by 60°. Left: Nearest Neighbor, Middle: Bilinear, Right: Bicubic interpolation.

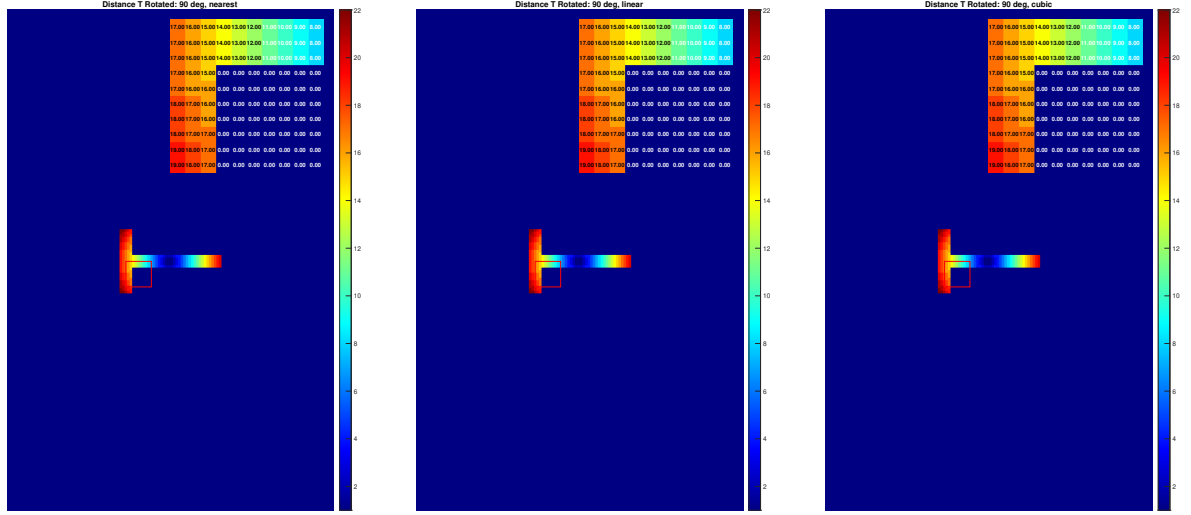


Figure 10: Distance-based 'T' rotated by 90°. Left: Nearest Neighbor, Middle: Bilinear, Right: Bicubic interpolation.

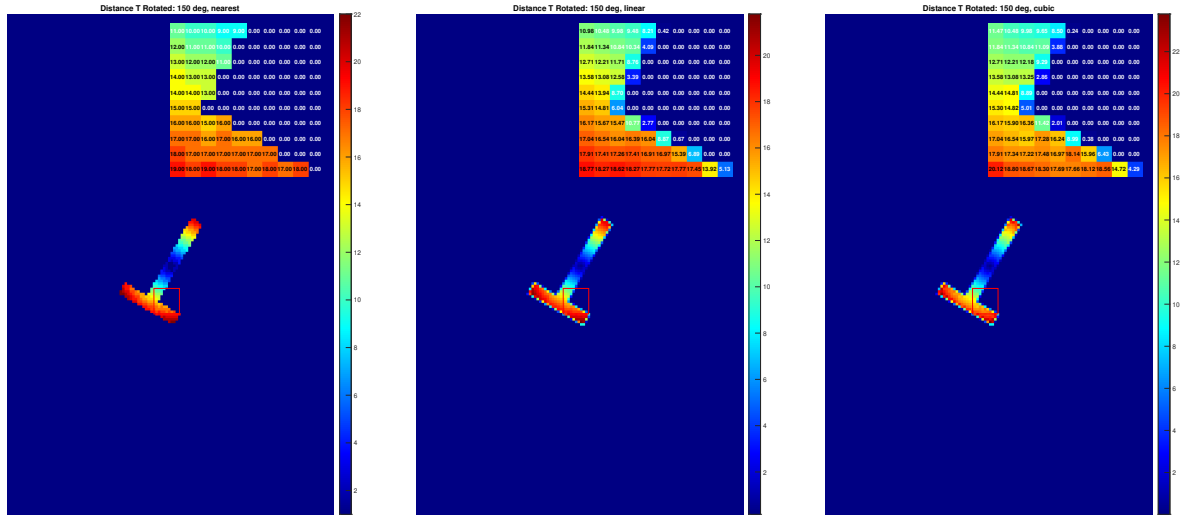


Figure 11: Distance-based 'T' rotated by 150°. Left: Nearest Neighbor, Middle: Bilinear, Right: Bicubic interpolation.

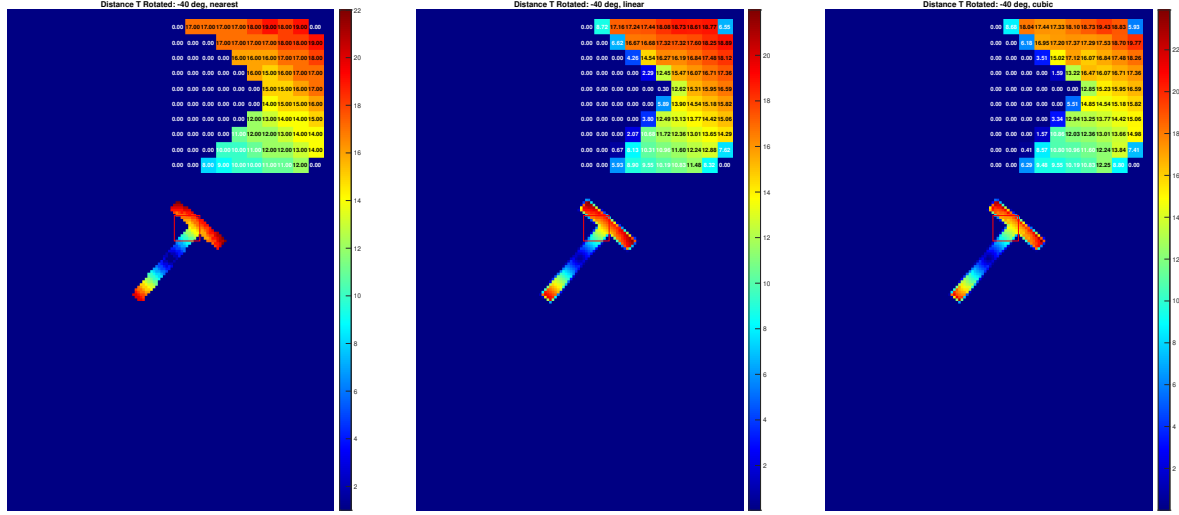


Figure 12: Distance-based 'T' rotated by -40° . Left: Nearest Neighbor, Middle: Bilinear, Right: Bicubic interpolation.